

## 一.資料庫功能說明

我做的是醫院病患資料的資料庫。

1.首先這個資料庫會有 6 個表格，也就是有 6 個實體。

### 1)patient:

這是拿來儲存病患的資料，包括病患的名字,生日,電話,地址,性別,信箱

其中，信箱是可以不填，而如果填了必須大家的信箱不得重複，性別只有男生和女生(生理性別)，主鍵為 patient\_id(這是由資料輸入給的編號)。

### 2)visit:

這是病患每次就診紀錄(每次為一行(一個 tuple))，就診時帶 patient\_id、日期、診斷。

### 3) Staff:

這是存醫護人員的資料，包括名字,電話,所擔任的角色。

### 4) Prescription:

存醫生開給病患的處方，會由外鍵 visit\_id 對每次的就診紀錄，紀錄當次的處方，裡

面的資料包括藥的名稱，多少劑量，什麼時候開的。

## 5) Tasks:

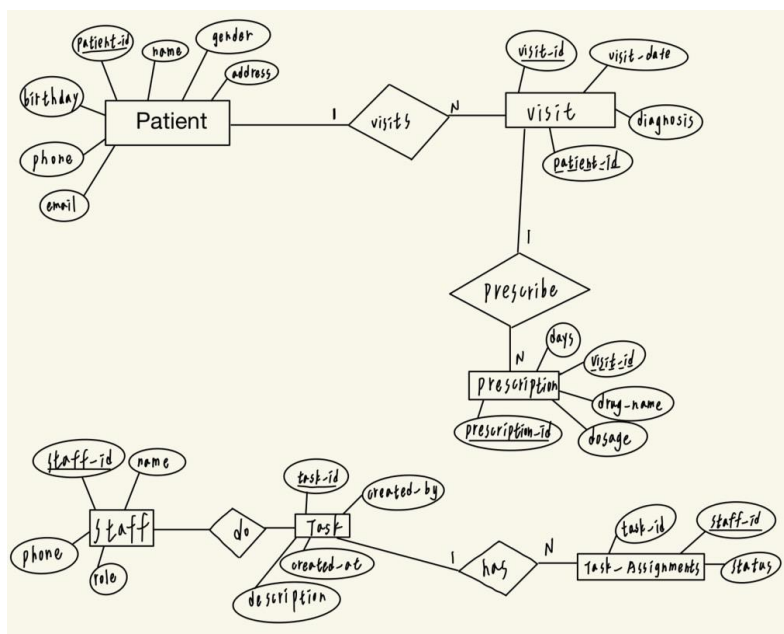
存代辦事項，裡面包刮由誰指派這個任務的，以及他何時指派的，還有任務內容。

## 6) Task\_Assignment:

任務的進度，裡面包刮我指派給誰任務，此任務目前的狀態，我給定三種狀態 -

> Pending ( 待處理 ) 或 In-Progress ( 進行中 ) 或 Done ( 完成 )

-這是此資料庫的 ER model:



## 二.可以提供的查詢

我目前設定 6 個可供介面查詢

\*因為想說 demo 方便 id 都是按照資料輸入的順序給的唯一編號，也可以改成身分證

等唯一識別碼。

## 1.病患基本資料

輸入病患 ID->可以查詢病患基本資料(包括名字,生日,電話,地址,性別,信箱)

如果是空值則為 Null 則不顯示

在 SQL server 的程式碼:

```
CREATE OR ALTER PROC sp_PatientInfo @pid INT
AS
SELECT *
FROM Patients
WHERE patient_id = @pid;
```

結果舉例:

一號病患:

| 輸入參數       |      |            |        |        |                    |         |      |
|------------|------|------------|--------|--------|--------------------|---------|------|
| 病患 ID:     | 1    | 員工 ID:     | 2      | 就診 ID: | 1                  | 年份:     | 1900 |
| patient_id | name | birthday   | gender | phone  | email              | address |      |
| 1          | 張三   | 1980-01-02 | M      |        | zhang3@example.com |         |      |

五號病患:

| 輸入參數       |      |            |        |              |                     |           |      |
|------------|------|------------|--------|--------------|---------------------|-----------|------|
| 病患 ID:     | 5    | 員工 ID:     | 2      | 就診 ID:       | 1                   | 年份:       | 1900 |
| patient_id | name | birthday   | gender | phone        | email               | address   |      |
| 5          | 郭五五  | 1993-07-15 | M      | 09-8125-4321 | kuofive@example.com | 台中市西屯區... |      |

2.輸入一個年份->可以找出這個年份後出生的病患

在 SQL server 的程式碼:

```
CREATE OR ALTER PROC sp_BornAfterYear @year INT
AS
SELECT patient_id, name, birthday
FROM Patients
WHERE birthday >= DATEFROMPARTS(@year,1,1);
```

結果舉例:

當我要找 1900 年後出生的人:

| 輸入參數       |      |            |      |
|------------|------|------------|------|
| 病患 ID:     | 5    | 員工 ID:     | 2    |
| 就診 ID:     | 1    | 年份:        | 1900 |
| patient_id | name | birthday   |      |
| 1          | 張三   | 1980-01-02 |      |
| 2          | 李四   | 1995-01-01 |      |
| 3          | 張三三  | 1988-03-04 |      |
| 4          | 李四四  | 2001-12-20 |      |
| 5          | 郭五五  | 1993-07-15 |      |
| 6          | 張六   | 1999-11-15 |      |
| 7          | 帳三   | 2000-01-01 |      |
| 8          | 張一   | 2000-04-05 |      |
| 9          | 王一   | 1999-05-07 |      |

3.找出姓王的病患:

在 SQL server 的程式碼:

```
CREATE OR ALTER VIEW v_SurnameWang AS
SELECT patient_id, name, phone
FROM Patients
WHERE name LIKE N'王%';
```

結果舉例:

會列出所有姓王的病患和他的聯絡電話

| patient_id | name | phone      |
|------------|------|------------|
| 9          | 王一   | 0956843256 |
| 10         | 王二   | 0986547896 |

#### 4.病患就診紀錄:

由輸入的病患 ID 可以找出該病患的就診紀錄和處方紀錄

在 SQL server 的程式碼:

```
CREATE OR ALTER PROC sp_PatientVisitDetail @pid INT
AS
SELECT v.visit_id, v.visit_date, v.diagnosis,
       p.drug_name, p.dosage, p.days
FROM Visits v
LEFT JOIN Prescriptions p ON p.visit_id = v.visit_id
WHERE v.patient_id = @pid
ORDER BY v.visit_date DESC;
```

結果舉例:

當我查完病患 2 號之後，我可以按下查詢他的就診紀錄跟處方，就可以知道他之前看的病以及醫生開給他的藥

| 輸入參數     |            |           |           |                   |
|----------|------------|-----------|-----------|-------------------|
| 病患 ID:   | 2          | 員工 ID:    | 2         | 就診 ID: 1 年份: 1900 |
| visit_id | visit_date | diagnosis | drug_name | dosage days       |
| 3        | 2025-06-15 | 頭痛        |           |                   |
| 6        | 2025-06-13 | 頭痛        | 藥物B       | 250 mg 5          |
| 2        | 2025-06-01 | 感冒        |           |                   |

#### 5.員工指派任務查詢:

輸入員工 ID 可以知道由此員工創建的任務

在 SQL server 的程式碼:

```
CREATE OR ALTER PROC sp_TasksByCreator @creator INT
AS
SELECT task_id, description, created_at
FROM Tasks
WHERE created_by = @creator;
```

結果舉例:

由員工 1 所指派的任務

| 輸入參數   |   |        |      |
|--------|---|--------|------|
| 病患 ID: | 2 | 員工 ID: | 1    |
| 就診 ID: | 1 | 年份:    | 1900 |

| task_id | description  | created_at |
|---------|--------------|------------|
| 1       | 處理病患 1 的就診資料 | 2025-06-13 |
| 3       | 病患 3 跟進回診    | 2025-06-13 |
| 4       | 處理病患 1 的就診資料 | 2025-06-13 |
| 6       | 病患 3 跟進回診    | 2025-06-13 |
| 7       | 處理病患 1 的就診資料 | 2025-06-13 |
| 9       | 病患 3 跟進回診    | 2025-06-13 |

## 6.就診處方內容

輸入就診 ID 就可以查詢該次就診的處方內容

在 SQL server 的程式碼:

```
CREATE OR ALTER PROC sp_VisitPrescription @vid INT
AS
SELECT prescription_id, drug_name, dosage, days
FROM Prescriptions
WHERE visit_id = @vid;
```

結果舉例:

就診 ID 為 5 的的處方內容，包刮開的藥物以及劑量，時間

輸入參數

病患 ID: 5 員工 ID: 1 就診 ID: 5 年份: 1900

| prescription_id | drug_name | dosage | days |
|-----------------|-----------|--------|------|
| 2               | 藥物A       | 500 mg | 7    |

## 7.提供加入病患資料

可以在介面上輸入病患基本資料，按增新後會存過去資料庫，如果再 SQL server 那邊選取所有資料就可以看到剛剛新增的病患。

舉例:

### 1.輸入病患資料:

新增病患

姓名 林六

生日 YYYY-MM-DD 2005-08-01

性別 M/F M

電話 0929587458

Email 633@gmail.com

地址 台南市善化區

新增

### 2.到 SQL server 看病患表格的資料，就可以看到剛剛新增加的(最下面)

|    | patient_id | name | birthday   | gender | phone        | email                     | address   |
|----|------------|------|------------|--------|--------------|---------------------------|-----------|
| 1  | 1          | 張三   | 1980-01-02 | M      | NULL         | zhang3@example.com        | NULL      |
| 2  | 2          | 李四   | 1995-01-01 | F      | NULL         | li4@example.com           | NULL      |
| 3  | 3          | 張三三  | 1988-03-04 | M      | 09-1234-5678 | damingch@example.com      | 台北市信義區... |
| 4  | 4          | 李四四  | 2001-12-20 | F      | 09-8765-4321 | xiaofang.chen@example.com | 高雄市左營區... |
| 5  | 5          | 郭五五  | 1993-07-15 | M      | 09-8125-4321 | kuofive@example.com       | 台中市西屯區... |
| 6  | 6          | 張六   | 1999-11-15 | F      | 0912645398   | six@gmail.com             | 嘉義市學府路    |
| 7  | 7          | 帳三   | 2000-01-01 | F      | 0915487951   | NULL                      | NULL      |
| 8  | 8          | 張一   | 2000-04-05 | M      | 0956821654   | one@gmail.com             |           |
| 9  | 9          | 王一   | 1999-05-07 | M      | 0956843256   | wang@gmail.com            | 台北市       |
| 10 | 10         | 王二   | 1888-07-08 | F      | 0986547896   | wangtwo@gmail.com         | 花蓮市       |
| 11 | 11         | 林六   | 2005-08-01 | M      | 0929587458   | 633@gmail.com             | 台南市善化區    |

## 二,應用程式開發環境說明+ 應用程式使用說明

使用 SQL server(表格+查詢)+python(介面設計+連線到 SQL server)

首先我是用 SQL server，因為上課交的語法可以直接實作，MySQL 有一些細節沒有學過，所以想說使用 SQL server。

1.SQL server(上課教過的語法):

我在這裡創建表格，以及增新一些 demo 的資料，還有建立可以讓 python 呼叫的查詢，因為在這裡面做比較能用上課所教的語法，所以先在這裡做這些。也可以從這裡去檢視資料庫裡的資料。

2.python(額外學習的語法):

\*要注意連線到的 SQL server 名稱

- Tkinter:設計顯示介面

- pyodbc: 建立與 SQL Server 的 連線

- SQL server: 存放程序，要定義位置(建置在 ClinicDB3 資料庫底下的 dbo 架構中)，可以提供直接呼叫

舉例:

sp\_PatientInfo @PatientID(@是傳參數)

傳入病患 ID，回傳該病患的基本資料(姓名、生日、性別、電話、Email、地址)

- SQL server view:

舉例:

v\_SurnameWang

簡化查詢，直接 `SELECT * FROM v_SurnameWang` 即可，不用每次都寫 `WHERE`

- 參數化查詢

舉例:

```
sql = "EXEC dbo.sp_PatientInfo ?"
```

```
params = (patient_id,)
```

```
cursor.execute(sql, params)
```

-> 就是對應到 SQL server 的對應查詢，傳參數進去做查詢

### 三. 正規化檢查

#### 1. 檢查是否符合 1NF

條件: 表格的每個欄位必須要是原子性的，每個欄位都只能儲存一個值，不能有重複組合。

在我的資料庫，六個表格的值都符合原子性，只會有一個值不會重複，所以符合。

#### 2. 檢查是否符合 2NF

條件: 每個非主鍵屬性都必須完全依賴於主鍵，消除部分依賴。

檢查:

舉例:

- 在 Visits 表中，visit\_id 是主鍵，所有的非主鍵屬性都完全依賴於 visit\_id，所以符合 2NF。

- 在 Task\_Assignment 表中，task\_id 和 staff\_id 是複合主鍵，並且所有其他欄位都依賴於複合主鍵，因此也符合 2NF。

而其他表格也是，因此符合 2NF。

### 3. 檢查是否符合 3NF

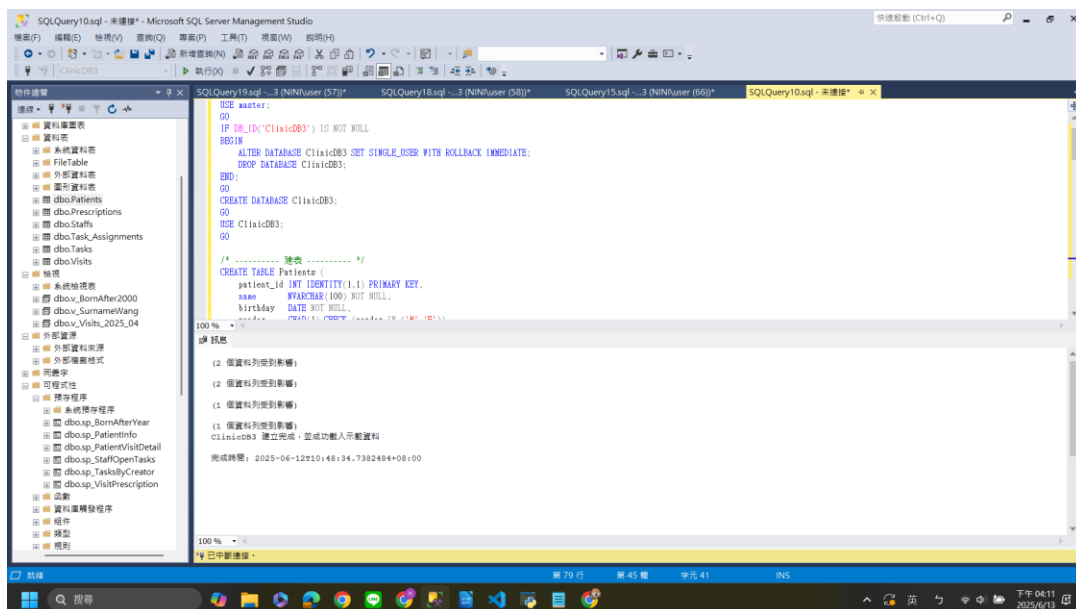
條件：去除**傳遞依賴**，一個非主鍵屬性不應該依賴於其他非主鍵屬性。

-在 **Tasks** 和 **Task\_Assignment** 表中，所有的非主鍵屬性都直接依賴於主鍵，並且沒有傳遞依賴。因此，它們符合 3NF。

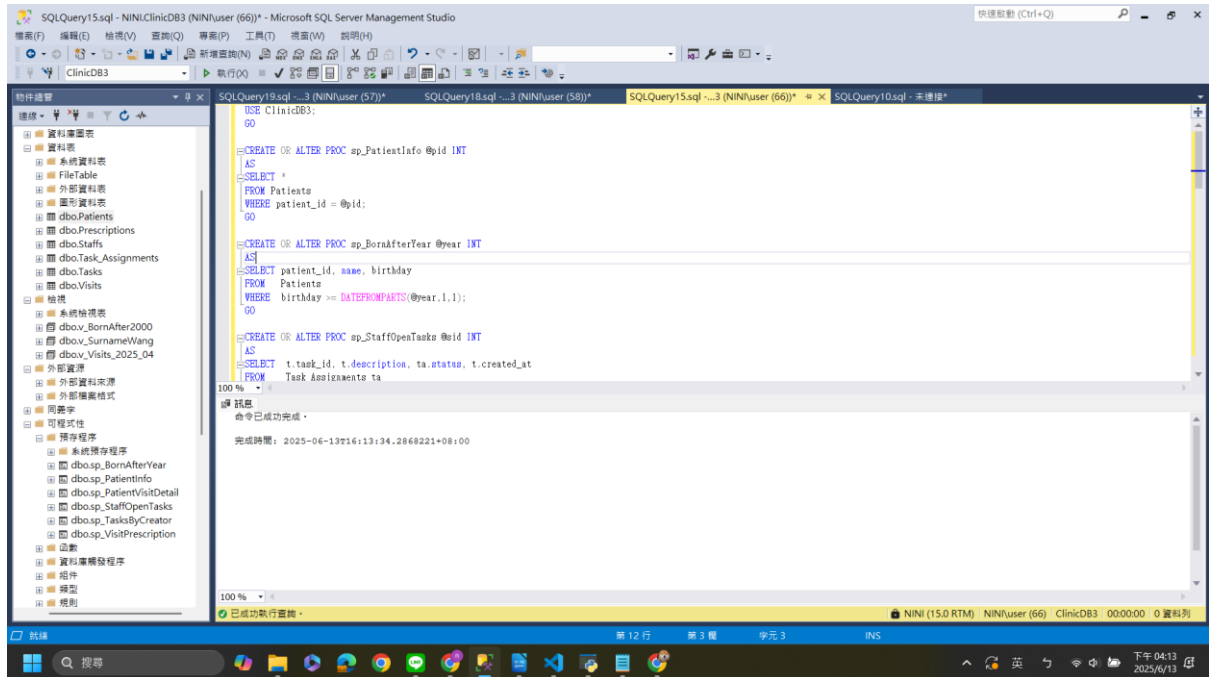
-**Patients** 和 **Visits** 表格符合 3NF，因為病患的基本資料不依賴於其他資料，而 **visit\_id** 也是唯一的。

## 四.demo 畫面

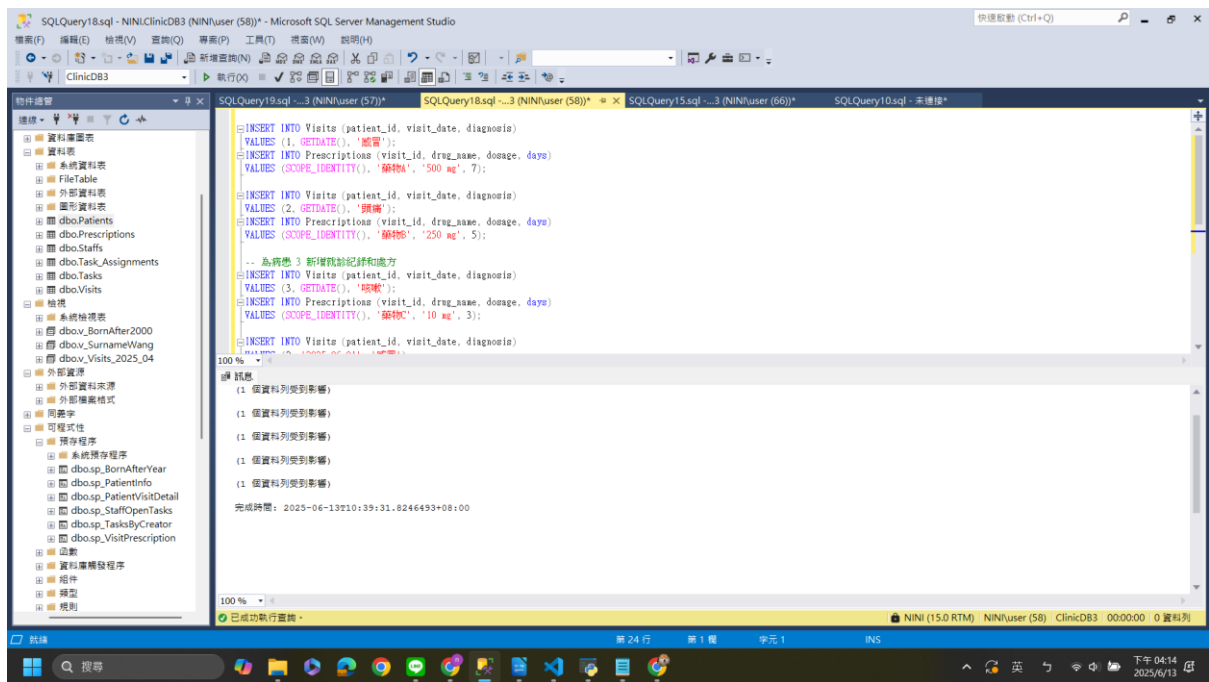
### 1.建立表格



### 2.創建查詢及 view

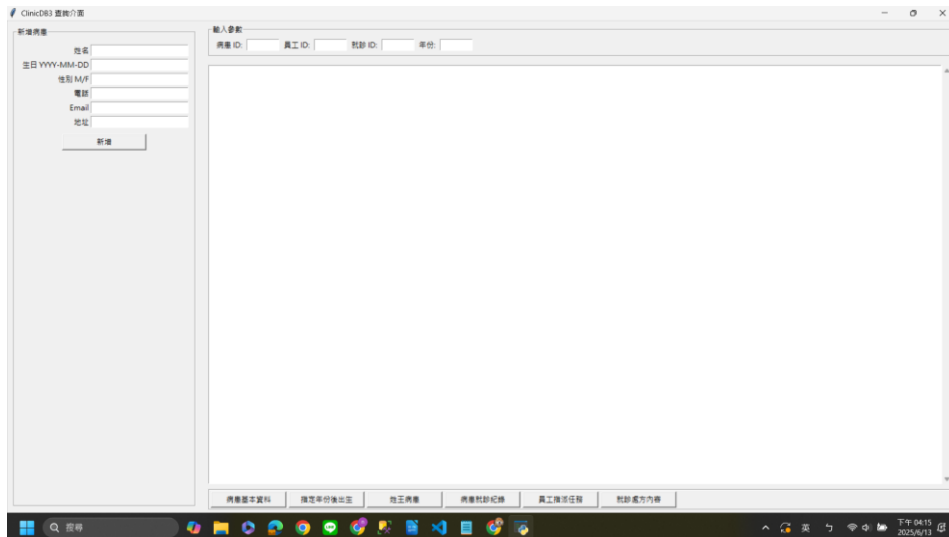


### 3. 增加資料



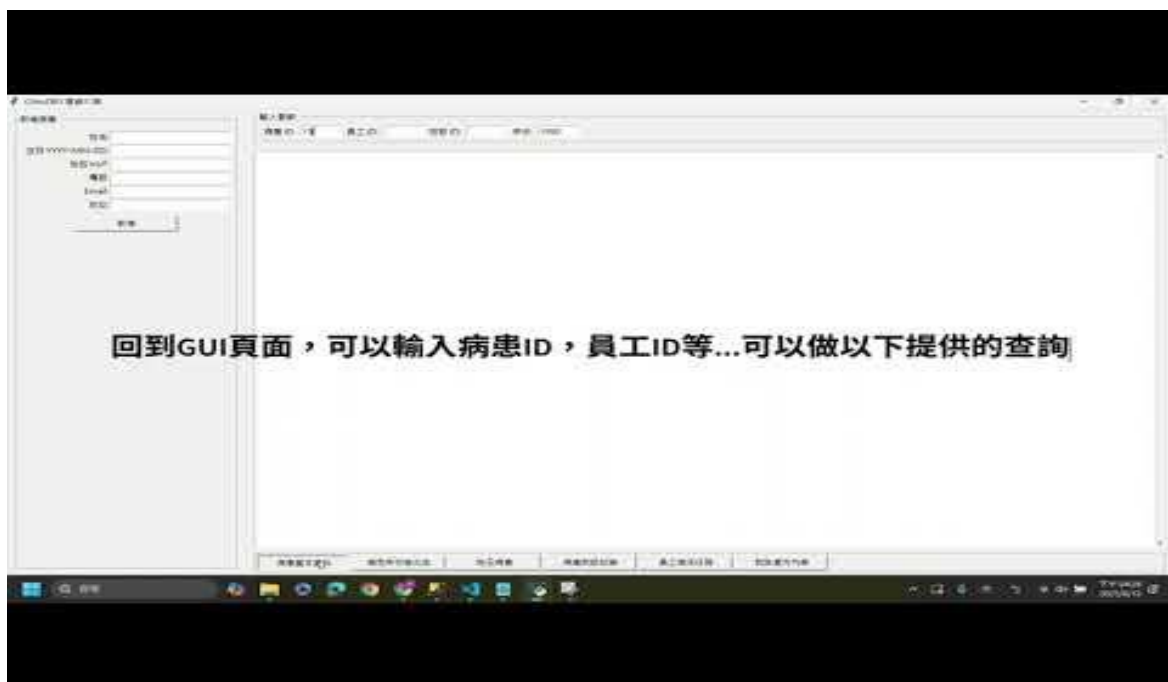
### 4. GUI

#### 1. 執行 python 畫面



## 2. 影片 demo

<https://youtu.be/9YkAPzqfKMQ>



附錄：

### 1. 創建表格所用程式碼

```
USE master; GO IF DB_ID('ClinicDB3') IS NOT NULL BEGIN ALTER
DATABASE ClinicDB3 SET SINGLE_USER WITH ROLLBACK IMMEDIATE; DROP
DATABASE ClinicDB3; END; GO CREATE DATABASE ClinicDB3; GO USE
ClinicDB3; GO
```

```
/* 建表 / CREATE TABLE Patients ( patient_id INT IDENTITY(1,1)
PRIMARY KEY, name NVARCHAR(100) NOT NULL, birthday DATE NOT NULL,
gender CHAR(1) CHECK (gender IN ('M','F')), phone NVARCHAR(20),
email NVARCHAR(255),
```

```
address NVARCHAR(255) ); / 這邊是因為如果填了 email 要是唯一，但病患可
以不一定要填 */ CREATE UNIQUE INDEX UQ_Patients_Email ON
Patients(email) WHERE email IS NOT NULL;
```

```
CREATE TABLE Staffs ( staff_id INT IDENTITY(1,1) PRIMARY KEY, name
NVARCHAR(100) NOT NULL, role NVARCHAR(50) NOT NULL, phone
NVARCHAR(20) );
```

```
CREATE TABLE Visits ( visit_id INT IDENTITY(1,1) PRIMARY KEY,
patient_id INT NOT NULL REFERENCES Patients(patient_id) ON DELETE
CASCADE, visit_date DATE NOT NULL, diagnosis NVARCHAR(255) ); CREATE
```

```
UNIQUE INDEX UQ_Visits_PatientDate ON Visits(patient_id,  
visit_date);
```

```
CREATE TABLE Prescriptions ( prescription_id INT IDENTITY(1,1)  
PRIMARY KEY, visit_id INT NOT NULL REFERENCES Visits(visit_id) ON  
DELETE CASCADE, drug_name NVARCHAR(255) NOT NULL, dosage  
NVARCHAR(100), days INT CHECK (days > 0) );
```

```
CREATE TABLE Tasks ( task_id INT IDENTITY(1,1) PRIMARY KEY,  
description NVARCHAR(255) NOT NULL, created_by INT REFERENCES  
Staffs(staff_id), created_at DATETIME DEFAULT GETDATE() );
```

```
CREATE TABLE Task_Assignments ( task_id INT NOT NULL REFERENCES  
Tasks(task_id) ON DELETE CASCADE, staff_id INT NOT NULL REFERENCES  
Staffs(staff_id), status NVARCHAR(20) CHECK (status IN  
( 'Pending', 'In-Progress', 'Done' )), PRIMARY KEY (task_id,  
staff_id) );
```

## 2. 鍵利查詢所用程式碼

```
USE ClinicDB3; GO
```

```
CREATE OR ALTER PROC sp_PatientInfo @pid INT AS SELECT * FROM  
Patients WHERE patient_id = @pid; GO
```

```
CREATE OR ALTER PROC sp_BornAfterYear @year INT AS SELECT  
patient_id, name, birthday FROM Patients WHERE birthday >=  
DATEFROMPARTS(@year,1,1); GO
```

```
CREATE OR ALTER PROC sp_StaffOpenTasks @sid INT AS SELECT t.task_id,  
t.description, ta.status, t.created_at FROM Task_Assignments ta JOIN  
Tasks t ON t.task_id = ta.task_id WHERE ta.staff_id = @sid AND  
ta.status <> 'Done'; GO
```

```
CREATE OR ALTER PROC sp_PatientVisitDetail @pid INT AS SELECT  
v.visit_id, v.visit_date, v.diagnosis, p.drug_name, p.dosage, p.days  
FROM Visits v LEFT JOIN Prescriptions p ON p.visit_id = v.visit_id
```

```
WHERE v.patient_id = @pid ORDER BY v.visit_date DESC; GO
```

```
CREATE OR ALTER PROC sp_TasksByCreator @creator INT AS SELECT  
task_id, description, created_at FROM Tasks WHERE created_by =  
@creator; GO
```

```
CREATE OR ALTER PROC sp_VisitPrescription @vid INT AS SELECT  
prescription_id, drug_name, dosage, days FROM Prescriptions WHERE  
visit_id = @vid; GO
```

```
CREATE OR ALTER VIEW v_SurnameWang AS SELECT patient_id, name, phone  
FROM Patients WHERE name LIKE N'王%'; GO
```

```
CREATE OR ALTER VIEW v_BornAfter2000 AS SELECT patient_id, name,  
birthday, gender FROM Patients WHERE birthday >= '2000-01-01'; GO
```

### 3. 增新 demo 資料所用程式碼

```
/*資料(有些病患資料是在 GUI 介面做的) */ INSERT INTO Patients  
(name,birthday,gender,email) VALUES (N'張三','1980-01-  
02','M','zhang3@example.com'), (N'李四','1995-01-  
01','F','li4@example.com');
```

```
INSERT INTO Staffs (name,role,phone) VALUES (N'張醫師  
, 'Doctor', '0912-000-001'), (N'李護理師', 'Nurse', '0912-000-002');
```

```
INSERT INTO Visits (patient_id,visit_date,diagnosis) VALUES  
(1,'2025-06-10',N'肺炎');
```

```
INSERT INTO Prescriptions (visit_id,drug_name,dosage,days) VALUES  
(1,N'Acetaminophen',N'500 mg',3);
```

```
INSERT INTO Visits (patient_id, visit_date, diagnosis) VALUES (2,  
'2025-06-01', '感冒'), (2, '2025-06-15', '頭痛');
```

```
INSERT INTO Visits (patient_id, visit_date, diagnosis) VALUES (3,  
'2025-06-05', '咳嗽');
```

```
INSERT INTO Visits (patient_id, visit_date, diagnosis) VALUES (1,
GETDATE(), '感冒'); INSERT INTO Prescriptions (visit_id, drug_name,
dosage, days) VALUES (SCOPE_IDENTITY(), '藥物 A', '500 mg', 7);
```

```
INSERT INTO Visits (patient_id, visit_date, diagnosis) VALUES (2,
GETDATE(), '頭痛'); INSERT INTO Prescriptions (visit_id, drug_name,
dosage, days) VALUES (SCOPE_IDENTITY(), '藥物 B', '250 mg', 5);
```

```
-- 為病患 3 新增就診紀錄和處方 INSERT INTO Visits (patient_id,
visit_date, diagnosis) VALUES (3, GETDATE(), '咳嗽'); INSERT INTO
Prescriptions (visit_id, drug_name, dosage, days) VALUES
(SCOPE_IDENTITY(), '藥物 C', '10 mg', 3);
```

```
INSERT INTO Tasks (description, created_by) VALUES ('處理病患 1 的就
診資料', 1), ('病患 2 預約檢查', 2), ('病患 3 跟進回診', 1);
```

```
INSERT INTO Task_Assignments (task_id, staff_id, status) VALUES (1,
1, 'Pending'),
```

```
(2, 2, 'In-Progress'), (3, 1, 'Pending');
```

#### 4.GUI 程式碼

```
import tkinter as tk from tkinter import messagebox, scrolledtext
import pyodbc from datetime import date
```

```
#連線 CONN_STR = ( "Driver={ODBC Driver 17 for SQL Server};"
"Server=NINI;"
```

```
"Database=ClinicDB3;" "Trusted_Connection=yes;" )
```

```
def get_conn(): return pyodbc.connect(CONN_STR, autocommit=True)
```

```
def to_int(txt: str, label: str): """字串→int；若空或非整數則警告並回傳
None""" txt = txt.strip() if not txt.isdigit():
messagebox.showwarning("輸入錯誤", f"{label} 必須是整數！") return
None return int(txt)
```

```

#GUI root = tk.Tk() root.title("ClinicDB3 查詢介面")
root.geometry("920x640")

#左側：新增病患表單 form = tk.LabelFrame(root, text="新增病患",
padx=10, pady=10) form.pack(side="left", fill="y", padx=10, pady=10)

labels = ["姓名", "生日 YYYY-MM-DD", "性別 M/F", "電話", "Email", "地
址"] entries = {} for i, text in enumerate(labels): tk.Label(form,
text=text).grid(row=i, column=0, sticky="e") ent = tk.Entry(form,
width=22) ent.grid(row=i, column=1, pady=2) entries[text] = ent

def add_patient(): vals = [e.get().strip() for e in
entries.values()] if not vals[0] or not vals[1]:
messagebox.showwarning("必填", "姓名與生日必填") return try: with
get_conn() as conn: conn.execute( ""INSERT INTO Patients
(name,birthday,gender,phone,email,address) VALUES (?,?,,?,?,?)""",
vals ) messagebox.showinfo("完成", "病患已新增") for e in
entries.values(): e.delete(0, tk.END) except Exception as e:
messagebox.showerror("錯誤", str(e))

tk.Button(form, text="新增", command=add_patient,
width=18).grid(row=len(labels), column=0, columnspan=2, pady=8)

#右側：查詢區 query = tk.LabelFrame(root, text="輸入參數", padx=6,
pady=6) query.pack(fill="x", padx=10, pady=6)

var_pid = tk.StringVar() var_sid = tk.StringVar() var_vid =
tk.StringVar() var_year = tk.StringVar()

fields = [("病患 ID:", var_pid), ("員工 ID:", var_sid), ("就診 ID:",
var_vid), ("年份:", var_year)]

for i, (lab, var) in enumerate(fields): tk.Label(query,
text=lab).grid(row=0, column=2i, padx=2, sticky="e") tk.Entry(query,
textvariable=var, width=7).grid(row=0, column=2i+1, padx=2)

#結果 result = scrolledtext.ScrolledText(root, font=("Consolas",

```

```

10)) result.pack(fill="both", expand=True, padx=10, pady=8)

def run_query(sql: str, params=()): """執行查詢並把結果印到 result"""
print(f"DEBUG >> {sql} | Params: {params}")

if any(p is None for p in params):
    result.delete("1.0", tk.END)
    result.insert(tk.END, "(請先輸入有效參數)\n")
    return

try:
    with get_conn() as conn:
        cur = conn.cursor()
        cur.execute(sql, params)
        rows = cur.fetchall()
        headers = [d[0] for d in cur.description]
except Exception as e:
    messagebox.showerror("查詢錯誤", str(e))
    result.delete("1.0", tk.END)
    result.insert(tk.END, f"查詢錯誤：{e}\n")
    return

result.delete("1.0", tk.END)
if not rows:
    result.insert(tk.END, "(查無資料)\n")
    return

# 處理 None 值
rows = [
    tuple('' if v is None else (v.strftime('%Y-%m-%d') if
isinstance(v, date) else v) for v in r)
    for r in rows
]

```

```

col_widths = [
    max(len(str(r[i])) for r in rows + [headers]) for i in
range(len(headers))
]
fmt = " ".join(f"{{: <{w}}}" for w in col_widths) + "\n"
result.insert(tk.END, fmt.format(*headers))
result.insert(tk.END, "-" * (sum(col_widths) + 2 * len(col_widths))
+ "\n")
for r in rows:
    result.insert(tk.END, fmt.format(*r))


btns = [ ("病患基本資料", lambda: run_query("EXEC
dbo.sp_PatientInfo ?", (to_int(var_pid.get(), "病患 ID"),))), ("指定
年份後出生", lambda: run_query("EXEC dbo.sp_BornAfterYear ?",
(to_int(var_year.get(), "年份"),))), ("姓王病患", lambda:
run_query("SELECT * FROM v_SurnameWang")), ("病患就診紀錄", lambda:
run_query("EXEC dbo.sp_PatientVisitDetail ?", (to_int(var_pid.get(),
"病患 ID"),))), ("員工指派任務", lambda: run_query("EXEC
dbo.sp_TasksByCreator ?", (to_int(var_sid.get(), "員工 ID"),))), ("
就診處方內容", lambda: run_query("EXEC dbo.sp_VisitPrescription ?",
(to_int(var_vid.get(), "就診 ID"),))), ]

bar = tk.Frame(root, relief="groove", bd=2) # 按鈕區
bar.pack(fill="x", padx=10, pady=(0, 4))

for i, (txt, cmd) in enumerate(btns): tk.Button(bar, text=txt,
command=cmd, width=16).grid(row=0, column=i, padx=2, pady=2)

root.mainloop()

```

